

Lab: 2D Convolution -- AXI-Stream & AXI-Lite Array

This notebook demonstrates how to interface with an HLS 2D Convolution IP. It showcases:

1. **AXI-Stream** via DMA for high-speed image data transfer.
2. **AXI-Lite** for scalar registers (rows , cols).
3. **AXI-Lite Memory Map** for passing a 3x3 kernel array.

1. Load Overlay

```
In [1]: import time
import struct
import numpy as np
import matplotlib.pyplot as plt
from pynq import Overlay, allocate
from scipy.signal import convolve2d

ol = Overlay("design_1_wop.bit")
print("Overlay loaded")
print("IP blocks:", list(ol.ip_dict.keys()))

dma      = ol.axi_dma_0
conv2d    = ol.conv2d_stream_0 # FIXED: Updated to match Vivado IP name

Overlay loaded
IP blocks: ['conv2d_stream_0', 'axi_timer_0', 'axi_dma_0', 'processing_system7_0']
```

2. Prepare Test Image & Kernel

We will generate a simple synthetic image and use a standard 3x3 Edge Detection kernel.

Note: MAX_WIDTH is set to 64 in the HLS code, so cols must be ≤ 64 .

```
In [2]: from PIL import Image
import numpy as np

ROWS = 64
COLS = 64

# 1. Load the image file (replace 'your_image.jpg' with your actual file path)
img = Image.open("cat.jpg")

# 2. Convert the image to grayscale ('L' stands for Luminous/grayscale)
img_gray = img.convert('L')

# 3. Resize the image to 64x64 to match the MAX_WIDTH constraint of your HLS I
img_resized = img_gray.resize((COLS, ROWS))

# 4. Convert to a numpy array and cast to float32
# Dividing by 255.0 normalizes the pixel values to a range between 0.0 and 1.0
# which matches the format of your previous synthetic image.
image_in = np.array(img_resized, dtype=np.float32) / 255.0

# 3x3 Edge Detection Kernel
kernel_3x3 = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
], dtype=np.float32)

print(f"Image size: {ROWS}x{COLS}, Kernel:\n{kernel_3x3}")
```

```
Image size: 64x64, Kernel:
[[-1. -1. -1.]
 [-1.  8. -1.]
 [-1. -1. -1.]]
```

3. Configure IP (AXI-Lite)

We must write `rows` and `cols` to their respective scalar registers. For the `kernel` array, Vitis HLS maps it to a memory block on the AXI-Lite interface. We must pack Python floats into 32-bit unsigned integers and write them sequentially.

(Check `xconv2d_stream_hw.h` in your Vitis HLS export to confirm these exact offsets)

```
In [3]: CTRL_REG      = 0x00
        ROWS_OFFSET   = 0x10
        COLS_OFFSET   = 0x18
        KERNEL_BASE   = 0x40 # FIXED: Updated to 0x40 based on the .hwh file

def float_to_uint(f):
    return struct.unpack('<I', struct.pack('<f', f))[0]

# 1. Write dimensions
conv2d.write(ROWS_OFFSET, ROWS)
conv2d.write(COLS_OFFSET, COLS)

# 2. Write 3x3 Kernel (9 elements, 4 bytes each)
kernel_flat = kernel_3x3.flatten()
for i, val in enumerate(kernel_flat):
    addr = KERNEL_BASE + (i * 4)
    conv2d.write(addr, float_to_uint(val))
```

4. Execute DMA Transfer

Data path: PS → DMA TX → conv2d IP → DMA RX → PS

```
In [4]: in_buf  = allocate(shape=(ROWS * COLS,), dtype=np.float32)
        out_buf = allocate(shape=(ROWS * COLS,), dtype=np.float32)

# Flatten 2D numpy array into 1D DMA buffer
np.copyto(in_buf, image_in.flatten())
out_buf[:] = 0

t0 = time.perf_counter()

# FIXED DMA ORDER: Setup the receiver BEFORE sending data to prevent deadlocks
dma.recvchannel.transfer(out_buf)
dma.sendchannel.transfer(in_buf)

# Start IP
conv2d.write(CTRL_REG, 0x01)

# Wait for transfers to complete
dma.sendchannel.wait()
dma.recvchannel.wait()

t_dma = time.perf_counter() - t0
print(f"Processed {ROWS*COLS} pixels in {t_dma*1e3:.2f} ms")

# Reshape output back to 2D
image_out_hw = np.array(out_buf).reshape((ROWS, COLS))
```

Processed 4096 pixels in 15.72 ms

5. Verification & Visualization

The HLS architecture reads the input stream but doesn't output valid calculated pixels until the line buffers fill up. Because of this lack of border padding, the outermost 1-pixel border will be invalid. We crop `[1:-1, 1:-1]` to compare the valid processing window.

```

In [6]: # Golden software model
image_out_sw = convolve2d(image_in, kernel_3x3, mode='same', boundary='fill',

hw_valid = image_out_hw[1:-1, 1:-1]
sw_valid = image_out_sw[1:-1, 1:-1]

max_diff = np.max(np.abs(hw_valid - sw_valid))
print(f"Max |HW - SW| (inner window) = {max_diff:.2e}")
if max_diff < 1e-4:
    print("PASS: Hardware output matches Software golden model!")
else:
    print("WARNING: Outputs differ.")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image_in, cmap='gray')
axes[0].set_title("Input Image")
axes[1].imshow(image_out_sw, cmap='gray')
axes[1].set_title("Software Output")
axes[2].imshow(image_out_hw, cmap='gray')
axes[2].set_title("Hardware Output")
SCALE=1
for ax in axes:
    ax.axis('off')
plt.show()
# Error heatmap
diff_map = np.abs(image_out_hw - image_out_sw)
fig2, ax2 = plt.subplots(figsize=(6, 5))
im = ax2.imshow(diff_map, cmap='hot')
ax2.set_title(f"|HW - SW| max={max_diff:.5f} 1LSB={1/SCALE:.5f}")
plt.colorbar(im, ax=ax2)
ax2.axis('off')
plt.tight_layout()
plt.show()
# — Error Heatmap —————
# Calculate the absolute difference using the valid inner window
diff_map = np.abs(hw_valid - sw_valid)

fig_heat, ax_heat = plt.subplots(figsize=(6, 5))

# Use the 'hot' colormap to highlight areas with the largest errors
im = ax_heat.imshow(diff_map, cmap='hot')

# Add a title showing the maximum error and the 1 LSB resolution
ax_heat.set_title(f"|HW - SW| Error Heatmap\nMax Error = {max_diff:.5f} (1 LS

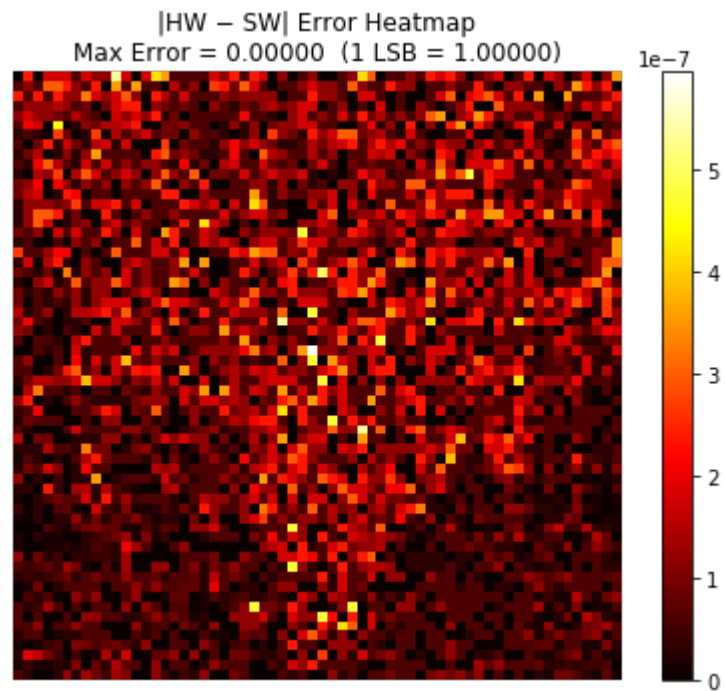
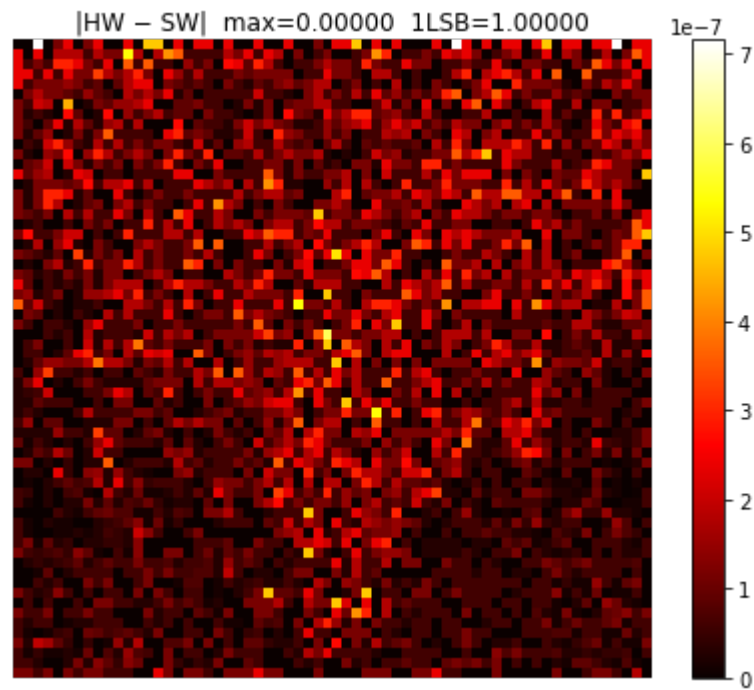
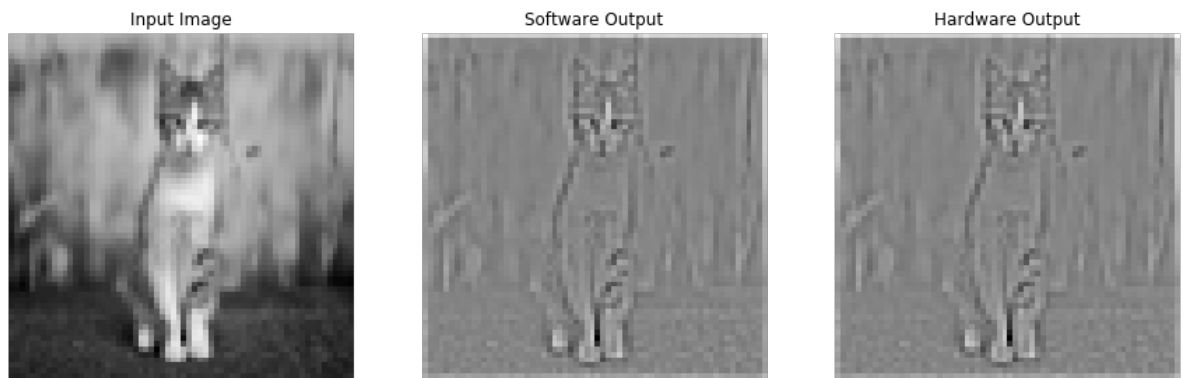
# Add the color scale bar
plt.colorbar(im, ax=ax_heat)
ax_heat.axis('off')

plt.tight_layout()
plt.show()

```

Max |HW - SW| (inner window) = 5.96e-07

PASS: Hardware output matches Software golden model!



6. Cleanup

```
In [ ]: in_buf.freebuffer()  
        out_buf.freebuffer()
```